Bridging Manual and Software-Executed Rule Systems in Tabletop Role-Playing Games

Torbjørn Økland

Master's thesis in Software Engineering at

Department of Computer science, Electrical engineering and Mathematical sciences,
Western Norway University of Applied Sciences

Department of Informatics,
University of Bergen

February 2026



Western Norway
University of
Applied Sciences



Abstract

Pending...

Keywords: tabletop role-playing games, digitalization, end-user development, modding, software architecture, virtual tabletops

Contents

Acronyms **6**

1 Introduction **7**

2 Background and Context **8**
2.1 TTRPGs 8
2.2 Digital Support in Contemporary TTRPG Play 9
2.3 Manual and Automated Rule Execution 9
2.4 Design Tension 10

3 Related Work **11**
3.1 Traditional Tabletop Role-playing Systems 11
3.2 VTTs and Digital Tooling 12
3.3 Rule Engines and Data-Driven Game Systems 12
3.4 Summary 12

4 Problem Definition and Goals **14**
4.1 Problem Statement 14
4.2 Design Challenges 15
4.3 Goals 15
4.4 Non-Goals and Scope Limitations 16

5 System Overview **17**
5.1 Conceptual Overview 17
5.2 User Roles 17
5.3 Rules, Engine, and Presentation 18
5.4 Session Structure and Workflow 18
5.5 Assumptions and Constraints 19

6 Requirements **20**
6.1 Functional Requirements 20
6.2 Non-Functional Requirements 21
6.3 Constraints 21

7 System Design and Architecture **22**
7.1 Architectural Overview 22
7.2 Rule Representation 23
7.3 Event-Driven Rule Execution 23

7.4 State Management 24
7.5 P2P Synchronization Model 24
7.6 GM Authority and Manual Intervention 25
7.7 Design Rationale and Tradeoffs 26

8 Implementation **27**
8.1 Platform and Technology Choices 27
8.2 Rule Storage and Representation 28
8.3 Rule Evaluation and Expression Execution 28
8.4 Engine and State Management 29
8.5 Networking and Synchronization 30
8.6 Presentation Layer and Tooling 31
8.7 Limitations of the Prototype 32

9 Evaluation Methodology **34**
9.1 Evaluation Goals 34
9.2 Participants 34
9.3 Test Setup 35
9.4 Procedure 35
9.5 Data Collection 35
9.6 Evaluation Criteria 36

List of Figures

7.1 High-level component architecture showing the separation between rule definitions, rule execution, state management, synchronization, and presentation. 23
7.2 Event-driven rule execution pipeline from player or GM action to state update and result presentation. 24
7.3 GM–authoritative P2P synchronization model. Player clients send action requests to the GM, who executes them and broadcasts authoritative state updates. 25

Acronyms

EUD End-User Development.

GM Game Master.

P2P Peer-to-Peer.

TTRPG Tabletop Role-Playing Game.

UI User Interface.

VTT Virtual Tabletop.

Chapter 1

Introduction

Tabletop Role-Playing Game (TTRPG) have traditionally been designed for pen-and-paper play, with rules and mechanics optimized for manual execution by players and a Game Master (GM). This design context has shaped most established systems, favoring mechanics that remain tractable without computational assistance. While this approach supports flexibility and social interaction, it also imposes practical limits on mechanical complexity. It places a significant organizational burden on participants, particularly in mechanically dense situations such as combat.

In contrast, modern digital role-playing games demonstrate that complex, interdependent mechanical systems can remain accessible when supported by software automation. Existing digital tools for TTRPGs, commonly referred to as Virtual Tabletop (VTT)s, primarily aim to facilitate traditional play by digitizing character sheets, dice rolls, and maps, while leaving authoritative rule enforcement and comprehensive state management to the players and the GM. As a result, these tools improve convenience but do not fundamentally change the design space of tabletop rule-sets.

This thesis argues that closer integration between rule-set design and software architecture enables a different class of tabletop role-playing systems. By treating automation as a first-class design assumption, it becomes possible to support deeper and more expressive mechanics without increasing the cognitive or organizational burden on participants, while also improving pacing, consistency, and accessibility. The goal of this work is to design and evaluate a software-first tabletop role-playing system in which rules, state management, and user interaction are co-designed with automation in mind. From a software engineering perspective, the central contribution is the design and prototype implementation of a rule-driven architecture that remains robust under rule changes, maintains consistent shared state in a Peer-to-Peer (P2P) setting, and supports complex mechanical interactions while preserving the GM 's authority. The system is evaluated through play-testing with players and GMs, focusing on gameplay pacing, perceived complexity, learning curve, and the impact of automation on GM workload.

Chapter 2

Background and Context

This chapter provides the conceptual and technical background necessary to situate this work. It introduces the fundamental structure and design constraints of TTRPGs, discusses the role of digital tools in contemporary play, and contrasts manual and automated rule execution in interactive systems.

2.1 TTRPGs

TTRPG are collaborative games in which a group of players assumes the roles of fictional characters within a shared narrative world. One participant, commonly referred to as the GM, is responsible for presenting the setting, portraying non-player characters, and adjudicating the rules. In contrast, the remaining participants control individual player characters and describe their actions within the fictional context. The outcome of these actions is determined by a combination of predefined rules and random elements, typically involving dice rolls [1, 8].

An important characteristic of TTRPGs is that learning and playing them are not limited to understanding formal rules. New players must also acquire implicit social conventions, group-specific practices, and expectations about play style, often through observation and participation rather than direct instruction [8]. This social and interpretative dimension contributes both to the flexibility of tabletop play and to its learning curve.

Historically, TTRPGs have been designed for pen-and-paper play. Character states, inventories, abilities, and ongoing effects are tracked manually, and the players and the GM perform rule resolution. This design context strongly influences the structure of most rule sets. Mechanics must remain understandable, executable, and verifiable by humans during live play, often under time pressure and with limited attention. As a result, many systems favor relatively simple resolution models, coarse abstractions, and a limited number of simultaneously active effects [1].

These constraints represent a deliberate design tradeoff. Pen-and-paper systems prioritize flexibility, social interaction, and human interpretability. At the same time, they impose practical limits on the depth of mechanical interaction that

can be used reliably in play. As the number of conditional rules, triggered effects, or persistent states increases, the cognitive and organizational burden on the group grows accordingly, with the GM typically bearing most of this load [8, 1].

2.2 Digital Support in Contemporary TTRPG Play

With the widespread availability of personal computers and mobile devices, many groups now use digital tools to support tabletop play. These tools range from simple digital character sheets and dice rollers to full-featured VTTs that provide shared maps, token management, communication features, and integrated rule references. Such systems are commonly motivated by the desire to support remote play and to enhance immersion and convenience [10].

Most existing VTTs focus on digitizing established workflows rather than fundamentally changing them. Character sheets become interactive forms, dice rolls are automated, and maps are shared on screen. Still, the responsibility for interpreting rules, applying modifiers, tracking conditions, and resolving complex interactions largely remains with the players and the GM. In this sense, these systems primarily act as facilitators rather than as authoritative executors of the game logic [10].

This approach preserves compatibility with existing rule sets and respects the GM 's authority and flexibility. At the same time, it means that the underlying design constraints of pen-and-paper systems remain in place. Mechanics must still be chosen such that they can be managed manually, even if some bookkeeping is assisted by software.

In parallel, digital game communities have demonstrated strong interest in extending and modifying existing systems through modding and end-user programming. Such practices show that users value the ability to adapt and extend rule systems and game content beyond their original design, and that flexible, user-modifiable systems can foster long-term engagement and experimentation [13].

2.3 Manual and Automated Rule Execution

In fully digital games and interactive systems, complex behavior is commonly governed by rule engines and data-driven models. Model-driven and rule-based approaches allow developers to describe system behavior at a higher level of abstraction and to execute these descriptions within a dedicated runtime environment [15]. Similarly, domain-specific modeling languages and domain-specific rule systems are used to represent and manage complex logic in a structured and extensible way [2, 6].

In such systems, the computer acts as the authoritative executor of the rules. This makes it possible to handle large numbers of interacting rules, event-driven behaviors, and persistent state changes with a level of consistency and reliability that would be difficult to achieve through manual execution alone. Complexity

is shifted from the user's cognitive load to the system's computational processes, enabling richer interaction models without necessarily increasing user effort [15].

By contrast, traditional TTRPGs rely on human participants to interpret and apply the rules during play. While this supports flexibility, improvisation, and social negotiation, it also constrains the practical complexity of mechanics that can be used reliably. As rule systems grow more intricate, the likelihood of forgotten conditions, missed triggers, and inconsistent application increases, and the organizational burden placed on the GM and players grows accordingly [8, 1].

2.4 Design Tension

The contrast between manual and automated rule execution highlights a fundamental design tension. Manual systems excel at social interaction and narrative flexibility but struggle with scale and mechanical density. Fully digital systems excel at managing complex states but typically operate without human arbitration.

This tension provides the conceptual context for the problem formulation in the following chapters, which examine how these differing approaches to rule execution and state management can be reconciled within a tabletop role-playing setting.

Chapter 3

Related Work

This chapter positions the presented work within the landscape of existing tabletop role-playing systems, digital support tools, and software approaches to rule-driven and data-driven game systems. The discussion is structured into three areas: traditional tabletop role-playing systems, VTTs and digital tooling, and rule-based and data-driven game architectures.

3.1 Traditional Tabletop Role-playing Systems

Established tabletop role-playing systems such as *Dungeons & Dragons*, *Fallout 2d20*, and *Walking Dead RPG* are designed primarily for pen-and-paper play and manual rule execution [14, 7, 3]. Their rule sets reflect the constraints of live, human-mediated play, emphasizing mechanics that can be resolved quickly and tracked with limited bookkeeping. Common design patterns include turn-based action resolution, coarse-grained defensive abstractions, and a limited number of simultaneously active conditions or triggered effects.

While these systems differ in tone and specific mechanics, they share a core assumption that players and the GM are responsible for interpreting and enforcing the rules. This supports flexibility and improvisation, but also constrains the practical complexity of mechanics that can be used reliably during play [1, 8]. As a result, highly granular state models, long chains of conditional effects, or densely interconnected subsystems are often simplified or avoided.

Several systems attempt to increase mechanical depth through modular subsystems such as feat trees, perk systems, or extended condition lists. However, in practice, the usability of these systems still depends on the group's ability to remember and correctly apply the relevant rules, which can increase cognitive load and lead to selective omission or simplification during play [8]. These observations highlight the design space explored by this thesis, namely systems in which manual rule enforcement is no longer the primary limiting factor.

3.2 VTTs and Digital Tooling

VTTs and related digital tools represent the most common form of software support for TTRPGs. Such systems typically provide shared maps, token management, communication features, digital character sheets, and integrated dice rolling, with the primary goal of supporting remote play and reducing manual overhead [10].

Despite these features, most VTTs are intentionally designed to remain system-agnostic or to mirror existing pen-and-paper workflows closely. Rule interpretation, condition tracking, and resolving complex interactions are usually still handled by the players and the GM. In this sense, these tools primarily act as facilitators rather than as authoritative executors of game logic [10].

There also exist more specialized tools that provide deeper automation for specific systems, for example, through scripted character sheets or rule-specific plugins. However, these solutions are typically tightly coupled to a particular rule set and often rely on ad hoc scripting or manual configuration. They rarely provide a general, extensible architecture for defining and enforcing rules in a data-driven manner, and the underlying system's structure often constrains their automation capabilities.

3.3 Rule Engines and Data-Driven Game Systems

Outside the tabletop domain, many digital games and interactive systems use rule engines or data-driven architectures to define game mechanics in a flexible, extensible way. In such systems, behavior is often specified through declarative models, event-driven rules, or domain-specific languages, allowing designers to modify or extend mechanics without changing the core engine [15, 2, 6].

These approaches demonstrate that complex, interdependent mechanics can be managed systematically when rule execution is delegated to software. They also emphasize the importance of explicit state management, deterministic resolution, and structured rule representations for maintaining consistency and debuggability in large systems [15].

Related work on End-User Development (EUD) and modding further shows that users value the ability to adapt and extend systems without low-level programming, and that exposing structured rule or behavior representations can support such practices [13, 11, 9]. However, most of this work assumes a fully digital context in which the computer is the sole authority over the game state and outcomes.

3.4 Summary

In summary, traditional tabletop systems prioritize human interpretability and flexibility but are constrained by the need for manual rule enforcement [1, 8]. VTTs primarily focus on digitizing existing workflows without fundamentally

changing the role of software in rule execution [10]. Data-driven and rule-based game architectures in the digital domain demonstrate the feasibility and advantages of software-executed rules, but typically do so in contexts without human arbitration [15, 2, 6].

The approach presented in this thesis is situated at the intersection of these areas, combining ideas from rule-based and data-driven systems with the social, human-mediated context of TTRPGs.

Chapter 4

Problem Definition and Goals

The preceding chapters described the characteristics of TTRPGs, existing digital support tools, and rule-based, data-driven software systems. This chapter formulates the engineering problem addressed by this thesis and defines the goals and scope of the proposed solution.

4.1 Problem Statement

Traditional tabletop role-playing systems rely on players and the GM manually executing and enforcing rules. While this supports flexibility and improvisation, it imposes practical limits on the mechanical complexity that can be used reliably during play [1, 8]. As the number of interacting rules, conditional effects, and persistent states increases, the cognitive and organizational burden placed on participants grows, often leading to slow pacing, inconsistent rule application, and the omission or simplification of mechanics that are difficult to track in practice [8].

Existing digital tools, such as VTTs, partially reduce this burden by supporting tasks such as dice rolling, character sheet management, and map sharing. However, these tools typically do not assume responsibility for authoritative rule execution or comprehensive state management, leaving the core tasks of tracking conditions, applying modifiers, and resolving complex interactions to the players and the GM [10]. As a result, the fundamental design constraints of manual rule enforcement largely remain in place.

In contrast, rule-based and model-driven software systems demonstrate that complex, interdependent behavior can be executed consistently and efficiently when rules are represented as data and interpreted by a dedicated runtime engine [15, 2, 6]. This suggests that the limitations observed in tabletop systems are not inherent to the mechanics themselves, but rather to the requirement of manual execution. The central problem addressed by this thesis is therefore the absence of a cohesive architecture for tabletop role-playing systems that

integrates rule design and software execution in a way that treats automation as a first-class concern while remaining compatible with a human-moderated, social play context.

4.2 Design Challenges

Addressing this problem introduces several challenges. First, the system must support automated rule execution and persistent state management without obscuring the underlying game logic from players and the GM. Automation should reduce cognitive load rather than replace understanding or agency. Second, the system must remain flexible and modifiable, allowing GMs to adapt rules and content without programming, in line with established practices of end-user modification and extension [13, 11]. Third, the architecture must remain robust under rule changes and maintain a consistent shared state in a P2P setting, where no central authoritative server is assumed.

In addition, the system must balance increased mechanical expressiveness with accessibility. Greater mechanical complexity should not imply a proportionally steeper learning curve for new players, and the GM must retain the ability to intervene, override outcomes, and handle exceptional situations.

4.3 Goals

Based on these challenges, the following goals are defined for this work:

1. **Software-First Rule Design**
   Design a tabletop role-playing system in which the rule-set is intended to be executed and enforced by software, rather than merely supported by it.

2. **Automated State Management and Rule Resolution**
   Provide automated handling of combat resolution, condition tracking, triggered effects, and other mechanically dense subsystems to reduce manual bookkeeping and improve consistency.

3. **Flexibility and Modifiability**
   Enable GMs to modify rules, create homebrew content, and adjust system behavior without requiring changes to the core engine or programming knowledge, following principles of EUD [13, 11].

4. **Preservation of GM Authority**
   Ensure that the system supports manual intervention, overrides, and corrections, and does not enforce automation in a way that restricts the GM 's role.

5. **Robustness in a P2P Setting**
   Design an architecture that maintains consistent shared state across multiple clients, remains stable under rule changes, and supports deterministic resolution where appropriate.

6. **Prototype Implementation and Evaluation**
   Implement a prototype system to demonstrate the approach's feasibility,

and evaluate it through play-testing with players and GMs.

4.4 Non-Goals and Scope Limitations

To keep the project feasible within the scope of a master's thesis, several aspects are explicitly excluded. These include monetization and distribution concerns, high-fidelity graphics or audio, matchmaking, large-scale online infrastructure, and artificial intelligence-driven non-player characters. The prototype developed in this work focuses on demonstrating the architectural feasibility of software-first rule design and automated rule execution rather than on delivering a complete, production-ready system. Many usability refinements, content expansions, and extended tooling features are therefore intentionally deferred and discussed as directions for future work.

The following chapters build on these goals by presenting an overview of the proposed system and then detailing its requirements, architecture, and implementation.

Chapter 5

System Overview

This chapter provides a high-level overview of the proposed system and its intended use. It introduces the main concepts, user roles, and structural assumptions that guide the detailed requirements and architectural design presented in the following chapters.

5.1 Conceptual Overview

The system presented in this thesis is a software-first tabletop role-playing platform in which the rule-set and the supporting software are designed as a single, integrated system. Rule execution, state management, and consistency checking are handled by software rather than performed manually by participants. This enables mechanics that would be difficult to manage reliably through manual processes alone, such as persistent multi-turn effects and event-driven abilities.

At the same time, the system is not intended to function as a conventional video game. A human GM remains responsible for narrative control, scenario design, and adjudication of exceptional situations. The software serves as an executor of formalized rules and a manager of game state, while allowing the GM to intervene, override outcomes, or apply manual changes as needed.

The system supports different levels of interaction with the underlying mechanics. Players can engage with a limited, presentation-oriented view of the rules. At the same time, more advanced users and GMs can access and modify the full rule definitions and system state through dedicated tools.

5.2 User Roles

The system distinguishes between two primary user roles: the GM and the players.

The GM is responsible for preparing and running the game session. This includes selecting or modifying rules, defining game content such as items and abilities, managing the narrative flow, and supervising play. The GM has full authority over the game state and can override automated outcomes or apply

manual changes. The system supports this role by providing facilities for inspecting, editing, and correcting the current state of the game.

The players control individual characters within the game world. They interact with the system primarily by selecting actions, targeting entities, or making narrative choices as prompted by the GM. The system resolves the mechanical consequences of these actions, updates the shared state, and presents the results. Players are not required to understand the full formal structure of the rule system to participate in play.

5.3 Rules, Engine, and Presentation

Conceptually, the system is structured into three main layers: rules, engine, and presentation.

The rules layer contains the formal definitions of game mechanics, including attributes, items, abilities, conditions, and their interactions. These definitions are represented in a data-driven form and are intended to be editable without modifying the core engine. This supports adaptation and extension of the rule set while keeping the execution model consistent.

The engine layer is responsible for interpreting the rules, managing the game state, resolving actions, and enforcing consistency. It processes events such as player actions or turn transitions, applies the relevant rule logic, and produces state changes and outcomes. This layer also handles synchronization between participants in a P2P setting.

The presentation layer provides the User Interface (UI) for both GMs and players. It displays relevant information, accepts user input, and communicates results. While this layer is essential for usability and accessibility, it is treated primarily as a means of interacting with the underlying system rather than as a focus of this work.

5.4 Session Structure and Workflow

A typical session begins with the GM preparing or selecting a rule set and game content. This may involve adjusting existing rules, adding new items or abilities, or configuring a specific scenario. Once the session starts, players connect to the GM 's instance in a P2P configuration and select or load their characters.

During play, participants alternate between narrative interaction and mechanically structured phases such as combat. When a mechanically relevant action is performed, the system resolves it according to the current rule set, updates the shared state, and presents the outcome to all participants. The GM may intervene at any time to adjust results or handle situations that fall outside the formalized rules.

5.5 Assumptions and Constraints

The system is designed under several practical constraints. It assumes a trust-based play environment typical of tabletop groups and therefore uses a P2P architecture without a central authoritative server. It targets a prototype level of completeness suitable for evaluation rather than a production-ready platform. While cross-platform support is a design goal, this work focuses on the architecture and rule-execution model rather than on platform-specific optimization or user-interface polish.

The next chapter derives concrete functional and non-functional requirements from this overview and from the goals defined earlier.

Chapter 6

Requirements

This chapter defines the functional and non-functional requirements of the proposed system. These requirements are derived from the problem statement and the system overview and serve as a basis for the subsequent architectural and implementation decisions.

6.1 Functional Requirements

- The system shall provide automated execution of game rules, including action resolution, effect application, character state updates, and management of turn-based sequences such as combat.
- The system shall support persistent tracking of conditions, effects, and other stateful mechanics across multiple turns and scenes, and apply and remove them according to the active rule set.
- The system shall support event-driven mechanics, such as abilities or perks that trigger in response to defined in-game events, and shall evaluate and execute these triggers automatically.
- The system shall provide mechanisms to define and edit game rules, including attributes, items, abilities, and effects, without requiring modification of the core engine code.
- The system shall allow the GM to inspect and modify the game state at any time, including overriding automated outcomes and applying manual corrections.
- The system shall support P2P multiplayer sessions in which multiple participants share a synchronized game state.
- The system shall provide a UI that allows players to perform actions and view relevant information without requiring full knowledge of the underlying rule definitions.

6.2 Non-Functional Requirements

- The system shall maintain consistency of the shared game state across all participating clients during a session. Divergent states shall be prevented, detected, and resolved.
- The system shall be robust to changes in the rule set. Modifications to rules or content definitions shall not render the system unusable, and invalid configurations shall be detected where possible.
- The system shall support the deterministic resolution of mechanically relevant actions, except where randomness is explicitly specified by the rule set, to ensure consistent synchronization across peers.
- The system shall be extensible, allowing additional mechanics, content types, or subsystems to be introduced without requiring fundamental changes to the architecture.
- The system shall remain usable for both experienced and inexperienced players. In particular, increased mechanical complexity shall not require a proportional increase in the amount of rule knowledge needed to participate.
- The system shall be suitable for use in a trust-based environment and shall not rely on a central authoritative server.

6.3 Constraints

- The system is developed as a prototype for a master's thesis. It is not required to meet production-level standards for performance, security, or UI polish.
- The system does not aim to provide features such as matchmaking, large-scale online play, monetization, or artificial intelligence-driven non-player characters.
- The focus of this work is on architectural feasibility and rule execution rather than on content completeness or audiovisual presentation.

The next chapter presents the system design and architecture that address these requirements.

Chapter 7

System Design and Architecture

This chapter describes the design and architecture of the proposed system. It explains how the requirements defined in the previous chapter are realized through structural components, data flow, and separation of responsibilities. The focus is on the architectural principles that enable automated rule execution, flexible rule definition, and consistent shared state in a P2P environment.

7.1 Architectural Overview

The system is structured as a modular, layered architecture that separates rule definition, rule execution, state management, networking, and presentation concerns. This separation supports flexibility under rule changes, maintainability, and clear boundaries of responsibility between components.

Figure 7.1 shows the high-level component structure of the system. The main components are:

- A rules component that contains formal definitions of game mechanics in a data-driven form.
- A rule execution engine that interprets these definitions and applies them to the current game state.
- A state management component that stores and updates the authoritative game state.
- A synchronization component that propagates authoritative state changes between participants.
- A presentation layer that provides UIs for players and the GM.

The rules component is intentionally decoupled from the execution engine. This allows rules, content, and mechanics to be modified or extended without changing the core engine logic, supporting flexible adaptation and homebrewing while keeping the execution model consistent.

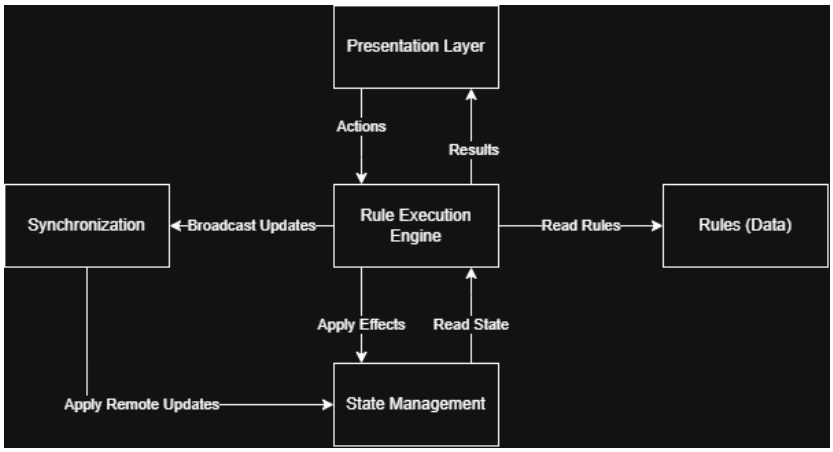


Figure 7.1: High-level component architecture showing the separation between rule definitions, rule execution, state management, synchronization, and presentation.

7.2 Rule Representation

Game rules are represented in a data-driven format rather than being hard-coded into the engine. This includes definitions of attributes, items, abilities, conditions, and their interactions. Each rule element is described using structured data that specifies its properties, triggers, and effects.

This representation serves several purposes. First, it allows GMs to edit and extend the rule-set without programming. Second, it enables validation and consistency checks on rule definitions before or during execution. Third, it ensures that the engine processes all mechanics uniformly, regardless of whether they originate from the base rule set or from user-defined extensions.

To prevent invalid or inconsistent configurations from destabilizing the system, rule definitions undergo structural and semantic checks. These checks verify, for example, that referenced attributes or effects exist, that triggers are well-formed, and that effect definitions satisfy basic consistency constraints.

7.3 Event-Driven Rule Execution

The core of the system follows an event-driven rule execution model. Player and GM actions, turn transitions, and state changes generate events that the engine processes. Rules may register handlers or triggers for specific event types, allowing abilities, perks, or conditions to react automatically when relevant situations occur.

Figure 7.2 illustrates the execution pipeline. When an action is performed, an event is created and passed to the rule engine. The engine selects the applicable rules, evaluates their conditions, and generates a set of effects. These effects are then applied to the game state, after which the updated state is propagated, and the results are presented to the participants.

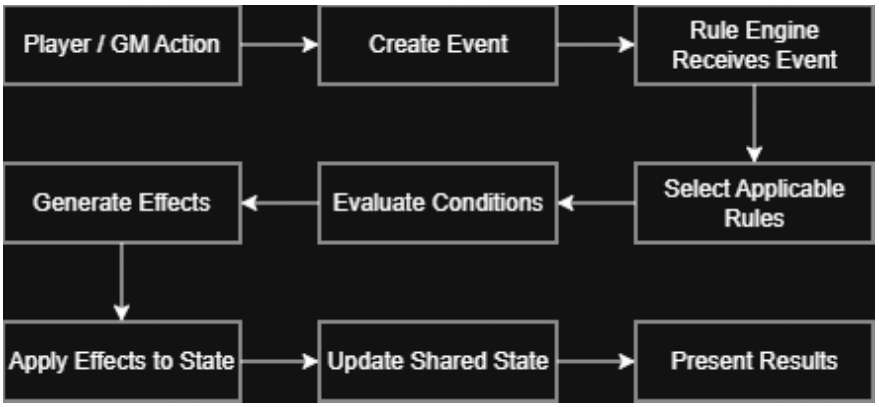


Figure 7.2: Event-driven rule execution pipeline from player or GM action to state update and result presentation.

This execution model allows complex interactions, such as abilities that trigger on hits, misses, status changes, or resource thresholds, without embedding special cases in the engine code. It also supports transparency and inspectability, as the sequence of applied rules and effects can be recorded or presented to explain how a particular outcome was produced.

7.4 State Management

The game state contains all mechanically relevant information, including character attributes, active conditions, turn order, and environmental effects. This state is maintained by a dedicated state management component that provides controlled access and modification operations.

All rule execution operates on this shared state representation. Direct, uncontrolled modification of the state is avoided to preserve consistency and traceability of changes. Instead, state updates are performed through well-defined effect applications generated by the rule engine.

This design supports deterministic resolution given identical inputs, facilitates synchronization across multiple clients, and allows the GM to inspect and modify the state in a controlled manner when manual intervention is required.

7.5 P2P Synchronization Model

The system follows a GM–authoritative P2P model, as illustrated in Figure 7.3. One participant, typically the GM, maintains the authoritative game state and executes all mechanically relevant actions using the rule engine. No central server infrastructure is assumed.

Actions initiated by the GM are executed locally on the authoritative instance, and the resulting state changes are broadcast to all participants. Actions initiated by players are sent to the GM as requests. The GM validates and executes

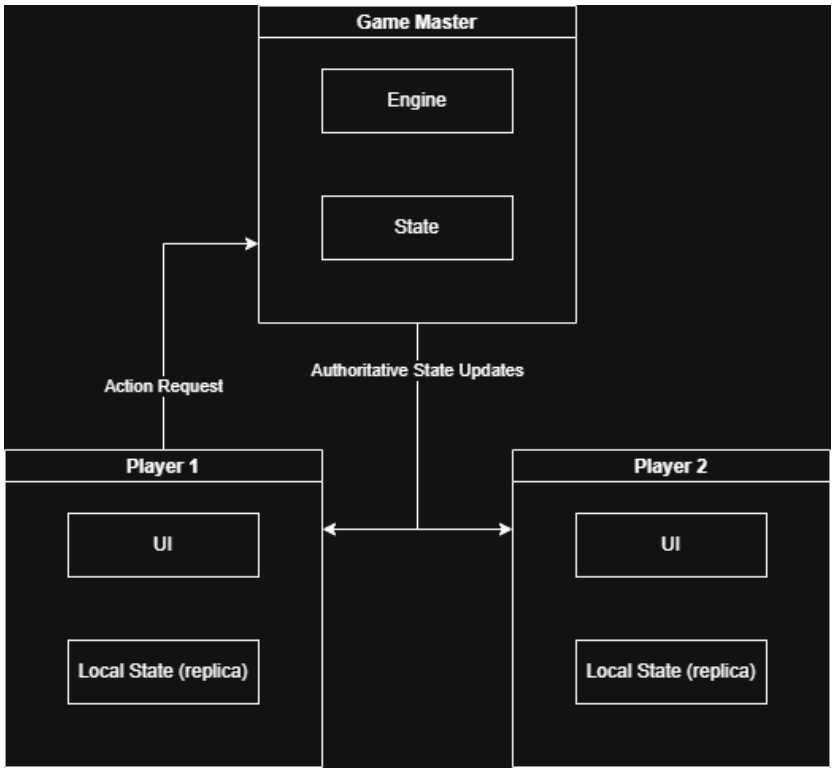


Figure 7.3: GM–authoritative P2P synchronization model. Player clients send action requests to the GM, who executes them and broadcasts authoritative state updates.

these requests, updates the authoritative state, and propagates the resulting state changes back to all clients.

Player clients maintain local replicas of the game state and apply the received authoritative updates in the order they are issued. This ensures consistent synchronization across participants while avoiding divergent rule execution. Since all authoritative decisions are made on the GM instance, this model simplifies consistency management and preserves GM's control over the session.

Basic mechanisms such as state versioning or checksums can be used to detect inconsistencies. While this approach does not aim to provide strong guarantees against malicious participants, it is sufficient for the assumed trust-based tabletop setting and simplifies deployment and operation.

7.6 GM Authority and Manual Intervention

A central design requirement is that the GM retains full authority over the game state. The architecture, therefore, includes explicit mechanisms for manual overrides and direct state edits. These interventions are treated as first-class state changes and are propagated to all participants in the same way as automated updates.

This ensures that exceptional situations, narrative-driven decisions, or corrections of mistakes can be handled without violating system consistency. It also prevents automation from becoming a rigid constraint on play, preserving the flexible and social nature of tabletop role-playing.

7.7 Design Rationale and Tradeoffs

The chosen architecture reflects several tradeoffs. Emphasizing data-driven rule definitions and event-based execution increases flexibility and expressiveness, but also introduces additional complexity in validation, debugging, and tooling. Relying on a GM — an authoritative P2P model — simplifies infrastructure requirements and consistency management, but limits the system's applicability to trust-based groups.

These tradeoffs are considered acceptable within the scope of this work, as the primary goal is to demonstrate the feasibility and benefits of software-first rule design in a tabletop context rather than to build a general-purpose or commercially robust platform.

The next chapter describes how this architecture is realized in a prototype implementation and discusses key implementation decisions and limitations.

Chapter 8

Implementation

This chapter describes the concrete realization of the architecture presented in the previous chapter. It focuses on the technologies used, the representation and execution of rules, state management and synchronization, and the tooling supporting the prototype. The goal is to document how the architectural concepts are mapped to a working system rather than to provide an exhaustive account of all implementation details.

8.1 Platform and Technology Choices

The prototype is implemented using the Godot game engine. A primary motivation for this choice is that Godot is open-source, which aligns with the long-term goal of enabling deep modification and extensibility beyond the scope of this thesis. Using an open-source engine allows technically inclined users to inspect, modify, and extend the application itself, reinforcing the concept of homebrewing not only at the rule level but also at the system level.

Godot provides a cross-platform runtime, a scene-based TTRPG system, networking facilities, and a scripting environment. The implementation uses GDScript to support rapid iteration and tight integration with the engine's tooling and asset pipeline.

Two Godot features are central to this project. First, the `Resource` system is used to represent and store game data, including rule definitions, character sheets, and campaign state [5]. This replaces the originally planned database-based approach and allows rules and content to be edited, inspected, versioned, and distributed using Godot's native workflow. Second, Godot's `Expression` system is used to represent and evaluate rule conditions and calculations at runtime [4]. This mechanism replaces the original plan to introduce a custom domain-specific language and enables user-editable rule logic without changes to the core engine code.

For networking, the prototype uses Godot's built-in networking facilities during development and local testing. For play-testing and distribution, the system integrates with Steam. Steam is used to provide P2P connectivity, storage of

save data, and application distribution [12]. In addition, Steam Remote Play is used as an early solution to enable mobile participation without requiring a dedicated mobile client. This setup reflects the trust-based, serverless assumptions of tabletop play and reduces the need for custom infrastructure within the scope of this work.

8.2 Rule Storage and Representation

Game rules and game data are represented using Godot's `Resource` system [5]. `Resources` are used to store rule definitions, character sheets, and campaign state in a structured, serializable format that integrates directly with the engine's asset pipeline. This approach replaces the originally envisioned database-based solution and offers a simpler, more flexible workflow for authoring, editing, and distributing content.

Each rule element, such as attributes, items, abilities, conditions, and effects, is represented as a dedicated resource type with a fixed schema. These resource types define the data required by the rule execution engine, including parameters, triggers, and effect specifications. Because resources are native Godot assets, they can be edited using the engine's inspector or custom editor tools, versioned alongside other project files, and loaded at runtime without additional infrastructure.

Using resources as the primary data format offers several advantages. First, it enables GMs to directly inspect and modify rules without requiring programming or database access. Second, it allows rule sets and content packs to be distributed and reused using the same mechanisms as other engine assets. Third, it ensures a uniform representation of both built-in and user-defined mechanics, as the engine treats all rule data identically regardless of its origin.

To ensure robustness, rule `resources` are validated when they are loaded. This validation includes structural checks, such as verifying that required fields are present, and referential checks, such as ensuring that referenced attributes, effects, or triggers exist. Invalid or incomplete rule definitions are reported through the GM interface and are not applied to the active rule set, preventing malformed data from destabilizing the system.

Beyond rules, the same resource-based approach is used to store character data and campaign state. This unifies the data model across the system and simplifies serialization, persistence, and synchronization. In the networking layer, state updates can be expressed as changes to these resource-backed structures, enabling consistent replication of authoritative state changes across clients.

8.3 Rule Evaluation and Expression Execution

To support data-driven and user-editable rule logic, the prototype uses Godot's `Expression` system [4] to evaluate conditions and compute effect values at runtime. `Expressions` are stored as part of the rule `resources` and are interpreted by the engine whenever relevant events are processed.

This mechanism allows rule authors to define conditional logic and calculations

without modifying engine code. For example, an ability may specify its activation condition as an expression over the current game state, or an effect may compute its magnitude based on character attributes and contextual parameters. At runtime, the engine constructs an evaluation context that includes relevant variables, such as character statistics, event parameters, and environmental values, and evaluates the expression in that context.

Using an interpreted `expression` system replaces the originally envisioned approach of introducing a dedicated domain-specific language. This reduces implementation complexity while still providing sufficient expressive power for the intended mechanics. It also leverages an existing, well-tested component of the engine, improving reliability and integration with the surrounding tooling.

Because expressions are evaluated at runtime and may be authored or modified by users, the implementation includes basic validation and error handling. Expressions are parsed when rule `resources` are loaded, and syntax errors are reported to the GM interface. During execution, evaluation errors or invalid results are caught and handled gracefully, preventing individual rule failures from terminating the session. This allows incomplete or experimental rules to be iterated on during play-testing without destabilizing the system.

Overall, the use of `Expression` provides a practical balance between flexibility and safety. It enables non-programmers to define and modify rule logic while keeping the execution model consistent and centrally controlled by the rule engine.

8.4 Engine and State Management

The game state is described as a centralized, authoritative structure manipulated by the rule execution engine and synchronized across participants. In the implementation, this conceptual state is realized through two primary synchronized data containers: `CampaignVariables` and `PlayerVariables`. Together, these structures represent the concrete form of the shared and per-player state discussed in the architectural design.

`CampaignVariables` exists on each client and represents the shared campaign-level state. It contains metadata such as the campaign name, description, and member list, as well as a `codex` field. The `codex` is a `Resource` that contains the rule definitions used by the system, corresponding to the rules component described in the architecture. All fields of `CampaignVariables`, including the `codex`, are synchronized across participants to ensure that all clients operate on a consistent campaign configuration.

Each player maintains a `PlayerVariables` instance containing the complete character sheet. This structure corresponds to the per-entity state managed by the state management component in the architectural model. `PlayerVariables` instances are synchronized with the GM, who maintains the authoritative copies used for resolving mechanically relevant actions. State changes resulting from rule execution or manual intervention are propagated back to the clients to keep all replicas consistent.

In addition to these state structures, the system provides a shared communica-

tion and reporting channel through a chat and an event log. This component serves as a lightweight, synchronized record of chat messages and skill check announcements, supporting transparency and shared session context without being part of the core mechanical state.

Combat state is represented using a synchronized list of enemy entities, with replication handled through Godot's multiplayer synchronization mechanisms, specifically `MultiplayerSpawner` and `MultiplayerSynchronizer`. This corresponds to the synchronized state layer described in the architectural model. During each combat round, players and enemies submit requests describing their intended actions. These requests are processed on the GM instance, which performs all rule evaluation and outcome determination, in accordance with the GM — authoritative execution model defined earlier.

The results of a combat round are communicated to all participants using an `Event Resource`. This resource represents the concrete form of the state updates produced by the rule execution engine in the architectural design. It contains the mechanically relevant consequences of the round, such as changes to health values, applied or removed conditions, and other state updates. Upon reception, each client applies the contents of the `Event` resource to its local state, ensuring that all participants remain synchronized while preserving the GM's authority over rule execution and state changes.

To preserve the GM's authority while maintaining a consistent and reproducible session state, the system maintains a chronological log of all `Event` resources generated during a session. When a session begins, the GM instance creates a snapshot copy of the initial campaign and player state. Each subsequent `Event` resource is appended to this log and represents a discrete state transition.

The GM interface allows previously generated `Event` resources to be edited. When such an edit occurs, the system reconstructs the current game state by replaying the entire sequence of `Event` resources starting from the initial snapshot. This mechanism ensures that manual corrections or narrative adjustments remain consistent with the event-driven execution model while preserving determinism and synchronization across clients. In this way, the GM can intervene and correct outcomes without breaking the underlying state model or desynchronizing participants.

8.5 Networking and Synchronization

The system follows a GM — authoritative P2P model. In this model, one participant, typically the GM, acts as the authoritative host responsible for resolving mechanically relevant actions and maintaining the authoritative game state, while other participants operate as clients that submit requests and receive state updates.

The implementation uses Godot's multiplayer framework for local development and testing, and integrates with Steam for play-testing and distribution. Steam is used to provide P2P connectivity, storage of save data, and application distribution. Also, it enables the use of Remote Play as an early solution for mobile participation. This setup avoids the need for dedicated server infrastructure

while remaining consistent with the trust-based assumptions of tabletop play.

Synchronization is organized around the core state structures introduced in the previous section. `CampaignVariables`, including the `codex` containing the rule definitions, are synchronized across all participants to ensure a consistent campaign configuration. Each player's `PlayerVariables`, which contain the complete character sheet, are synchronized with the GM, who maintains the authoritative copies used for rule execution. In addition, the shared chat and event log is synchronized to provide consistent communication and reporting across clients.

Combat-related entities, such as enemies, are replicated using Godot's multiplayer synchronization mechanisms, specifically `MultiplayerSpawner` and `MultiplayerSynchronizer`. This ensures that all participants observe the same set of combatants and their associated state.

Client interactions that affect the shared state are transmitted to the GM instance as requests. The GM processes these requests locally, evaluates the rules, updates the authoritative state, and broadcasts the resulting changes to all clients. For combat rounds, these updates are encapsulated in an `Event Resource` that contains the mechanically relevant outcomes, such as changes to health values, applied or removed conditions, and other state modifications. Clients apply these updates to their local replicas of the state, keeping all participants synchronized.

This approach centralizes rule execution and state mutation on the GM instance, simplifying consistency management and preserving the authority model defined in the architectural design. At the same time, it avoids the need for a central server and remains compatible with the social, trust-based setting typical of TTRPGs.

8.6 Presentation Layer and Tooling

The presentation layer provides the TTRPGs through which players and the GM interact with the system. In the architectural design, this layer is responsible for visualizing the current game state, collecting user input, and presenting the results of rule execution, while remaining decoupled from the core rule logic and state management. The implementation follows this separation by using Godot's scene and TTRPG system to realize distinct views and interaction workflows for the two primary roles.

For players, the interface focuses on interacting with their own `PlayerVariables`, selecting actions, and presenting outcomes. Players can view their character data, submit action requests, and receive updates reflecting authoritative state changes made by the GM instance. The interface also exposes the shared chat and event log, allowing players to follow communication and mechanically relevant announcements such as skill checks.

The GM interface provides additional functionality for inspecting and controlling the session. In addition to viewing all synchronized `CampaignVariables` and `PlayerVariables`, the GM can inspect the current campaign configuration, including the active `codex Resource` containing the rule definitions. The

interface also supports manual intervention in the game state, allowing the GM to correct errors or apply narrative-driven changes while keeping all clients synchronized.

Rule authoring and modification are supported through the use of Godot's `Resource` workflow. Because rules, character data, and campaign state are represented as resources, they can be edited using the engine's inspector and custom editor tools. This enables rapid iteration on mechanics and content without requiring changes to the core engine code. Errors detected during rule loading or `Expression` evaluation are reported through the interface, supporting debugging and iterative refinement of rule definitions.

Overall, the presentation layer serves as a thin interaction and visualization layer over the underlying engine and state management components. It exposes the system's functionality to users in a role-appropriate manner while preserving the separation between user interaction, rule execution, and state synchronization established in the architectural design.

In addition, the GM interface exposes the session's chronological `Event` log and allows individual `Event` resources to be inspected and edited. When an event is modified, the system reconstructs the current game state by replaying the event sequence from the session's initial state snapshot. This provides a controlled and transparent mechanism for correcting mistakes or applying narrative-driven changes while preserving the integrity of the synchronized state.

8.7 Limitations of the Prototype

The implemented system is a research prototype designed to evaluate the feasibility of software-first rule design and GM — authoritative automation — in a tabletop role-playing context. As such, several limitations are inherent to its current scope and level of maturity.

First, the implementation prioritizes correctness, flexibility, and inspectability over performance and scalability. No systematic performance optimization has been conducted, and the system is not intended to support large numbers of participants or high-frequency state updates. The networking model assumes a trust-based environment and does not address adversarial behavior, security hardening, or fault tolerance beyond basic consistency mechanisms.

Second, the TTRPG focuses on functionality rather than polish or accessibility. While the presentation layer exposes the necessary controls for players and the GM, usability refinements, visual design, and comprehensive onboarding support are outside the scope of this work.

Third, content completeness is limited. The rule-set and available mechanics are sufficient to demonstrate the architecture and execution model, but they do not constitute a full or balanced game system. In particular, parts of the combat workflow and supporting tooling are simplified compared to what would be required for a production system.

Finally, the implementation relies on specific platform features, such as Godot's `Resource` and `Expression` systems and the chosen networking integration. While

the architectural ideas are not tied to a particular engine, the prototype itself is not intended to be portable without additional engineering effort.

These limitations are a direct consequence of the project's prototype-oriented nature. The focus of this work is on demonstrating architectural feasibility and exploring the design space of software-first tabletop role-playing systems rather than delivering a production-ready platform.

Chapter 9

Evaluation Methodology

This chapter describes the methodology used to evaluate the proposed system. The goal of the evaluation is to assess whether a software-first, GM–authoritative tabletop role-playing system can support mechanically richer gameplay while maintaining usability, learnability, and manageable GM workload. The chapter defines the evaluation goals, the study setup, the participants, the procedure, and the data collection methods.

9.1 Evaluation Goals

The evaluation focuses on the following high-level goals:

- Assess whether automation and software-executed rules reduce the manual bookkeeping burden for the GM.
- Examine how the system affects gameplay pacing, particularly in mechanically dense situations such as combat.
- Evaluate the perceived complexity and learning curve for new and experienced players.
- Investigate whether the system preserves the GM's authority and flexibility in practice.
- Explore the usability of the rule authoring and modification workflow for homebrewing.

These goals are derived directly from the problem statement and the design objectives defined earlier in the thesis.

9.2 Participants

The evaluation is conducted with groups consisting of one GM and multiple players. Participants are recruited from existing tabletop role-playing groups and include both experienced and less experienced players. This mixed compo-

sition is intended to reflect realistic usage scenarios and to allow observation of differences in learning behavior and interaction with the system.

The GMs participating in the study have prior experience running tabletop role-playing sessions and are familiar with at least one existing tabletop system. Players vary in their level of prior experience with TTRPGs.

9.3 Test Setup

Each evaluation session is conducted using the prototype implementation described in the previous chapter. Sessions are run in a P2P configuration with the GM acting as the authoritative host. The same rule set and scenario structure are used across sessions to ensure comparability.

The test scenarios are designed to include both narrative interaction and mechanically structured phases, such as skill checks and combat encounters. This ensures that the evaluation covers both low-mechanics and high-mechanics gameplay situations.

All participants use the provided TTRPGs appropriate to their roles. The GM uses the GM interface with access to rule editing, state inspection, and manual intervention tools, while players use the player interface for character interaction and action submission.

9.4 Procedure

nn Each session follows a standardized procedure. Participants are first given a short introduction to the system and its basic interaction concepts. New players receive a brief explanation of the TTRPG and the core gameplay loop.

The session then proceeds through a prepared scenario that combines narrative and mechanically structured interactions. During play, the GM runs the session as usual, using the system to resolve actions and manage state. Players interact with the system by submitting actions and responding to outcomes.

After the session, participants are asked to provide feedback through a structured questionnaire and, where possible, a short discussion. This feedback focuses on usability, perceived complexity, pacing, and the impact of automation on the play experience.

9.5 Data Collection

Both qualitative and quantitative data are collected.

Quantitative data includes:

- Session duration and time spent in mechanically structured phases.
- Number of actions and rule resolutions performed by the system.
- Frequency of GM interventions and manual corrections.

Qualitative data includes:

- Player and GM feedback on usability and learnability.
- Perceived impact of automation on gameplay flow.
- Observations regarding confusion, errors, or friction during play.

In addition, the system's chat and event log are used to support post-session analysis of interaction patterns and rule resolution sequences.

9.6 Evaluation Criteria

The collected data is analyzed with respect to the evaluation goals defined at the beginning of this chapter. In particular, the analysis focuses on:

- Whether automation reduces manual workload for the GM.
- Whether gameplay pacing is improved or hindered by software-executed rules.
- How participants perceive the complexity and learning curve of the system.
- Whether the GM retains effective control and flexibility.
- How usable the rule modification and homebrewing workflow is in practice.

The results of this analysis are presented and discussed in the following chapter.

Bibliography

[1] Karl Bergström. "Creativity Rules: How Rules Impact Player Creativity in Three Tabletop Role-playing Games." In: *International Journal of Role-Playing* 3 (2012), pp. 4–17. DOI: 10.33063/ijrp.vi3.221. URL: https://journals.uu.se/IJRP/article/view/221 (visited on Oct. 1, 2025).

[2] Claudio Di Sipio et al. "On the use of LLMs to support the development of domain-specific modeling languages." In: *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*. MODELS Companion '24: ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems. Linz Austria: ACM, 2024, pp. 596–601. ISBN: 979-8-4007-0622-6. DOI: 10.1145/3652620.3687808. URL: https://dl.acm.org/doi/10.1145/3652620.3687808 (visited on Oct. 1, 2025).

[3] Free League Publishing. *The Walking Dead Universe Roleplaying Game Core Rulebook*. Stockholm, Sweden: Free League Publishing, 2023. ISBN: 9789189765207.

[4] Godot Engine Community. *Expression — Godot Engine Documentation*. Official documentation. 2025. URL: https://docs.godotengine.org/en/4.4/classes/class_expression.html (visited on Feb. 18, 2026).

[5] Godot Engine Community. *Resources — Godot Engine Documentation*. Official documentation. 2025. URL: https://docs.godotengine.org/en/4.4/tutorials/scripting/resources.html (visited on Feb. 18, 2026).

[6] Neel Mani, Markus Helfert, and Claus Pahl. "A Domain-specific Rule Generation Using Model-Driven Architecture in Controlled Variability Model." In: *Procedia Computer Science* 112 (2017), pp. 2354–2362. DOI: 10.1016/j.procs.2017.08.206. URL: https://www.sciencedirect.com/science/article/pii/S1877050917316022 (visited on Oct. 1, 2025).

[7] Modiphius Entertainment. *Fallout: The Roleplaying Game (2d20 System)*. London, UK: Modiphius Entertainment, 2020. ISBN: 9781912743681.

[8] Bryn Neuenschwander. *Playing by the Rules: Instruction and Acculturation in Role-Playing Games*. 2008. DOI: 10.2304/elea.2008.5.2.189. URL: https://journals.sagepub.com/doi/epdf/10.2304/elea.2008.5.2.189 (visited on Oct. 1, 2025).

[9] Christoph Neumann, Ronald A. Metoyer, and Margaret Burnett. "End-user strategy programming." In: *Journal of Visual Languages & Computing* 20.1 (2009), pp. 16–29. DOI: 10.1016/j.jvlc.2008.04.005.

URL: https://www.sciencedirect.com/science/article/pii/S1045926X08000281 (visited on Oct. 1, 2025).

[10] Anastasios N. Niarchos. "The use of Immersive VR in Tabletop Role Playing Games as a Virtual Tabletop." BSc Thesis. NATIONAL AND KAPODISTRIAN UNIVERSITY OF ATHENS, 2023. URL: https://pergamos.lib.uoa.gr/uoa/dl/object/3338579/file.pdf (visited on Jan. 10, 2025).

[11] Fabio Paternò and Carmen Santoro. "End-user development for personalizing applications, things, and robots." In: *International Journal of Human-Computer Studies* 131 (2019), pp. 120–130. DOI: 10.1016/j.ijhcs.2019.06.002. URL: https://www.sciencedirect.com/science/article/pii/S1071581919300722 (visited on Oct. 1, 2025).

[12] Valve Corporation. *Steamworks Documentation*. Official developer documentation. 2025. URL: https://partner.steamgames.com/doc/home (visited on Feb. 18, 2026).

[13] Matthew Wells. "Game not Over: End-User Programming and Game System Modding as Models for Extending Community Engagement." In: *ELPUB 2018*. Ed. by Leslie Chan and Pierre Mounier. Vol. Connecting the Knowledge Commons: From Projects to Sustainable Infrastructure. Toronto, Canada: ElPub, 2018. DOI: 10.4000/proceedings.elpub.2018.21. URL: https://hal.science/hal-01816719 (visited on Oct. 1, 2025).

[14] Wizards of the Coast. *Player's Handbook (Dungeons & Dragons, 5th Edition)*. Renton, WA: Wizards of the Coast, 2014. ISBN: 9780786965601.

[15] Enes Yigitbas et al. "Integrated model-driven development of self-adaptive user interfaces." In: *Software and Systems Modeling* 19.5 (2020), pp. 1057–1081. DOI: 10.1007/s10270-020-00777-7. URL: https://doi.org/10.1007/s10270-020-00777-7 (visited on Oct. 1, 2025).